

Petljak Lab Computational Tips and Tricks

Before you do anything

```
ssh into bigpurple (if unsure how, ask Luka)
run "echo 'umask 027' >> ~/.bashrc"
close and re-open your ssh connection
```

BigPurple

Documentation for BigPurple:

http://bigpurple-ws.nyumc.org/wiki/index.php/BigPurple_HPC_Cluster

<https://hpcmed.org/guide>

BigPurple is a compute cluster (a bunch of nodes (computers) strung together and networked) that we can submit jobs to in order to do computational things. It's structured by a few login nodes (bigpurple-ln[1-3]) that you automatically SSH into, and then you submit jobs to the compute nodes to do computational things. Managing workloads and priorities among the many many users of BigPurple is handled using SLURM (<https://slurm.schedmd.com/documentation.html>).

For temporary storage, the HPC guides say there is scratch space. In practice, there is no scratch space. Use /gpfs/data/petljaklab/ (make a subdirectory for yourself!) for your storage needs because scratch fills unpredictably and could cause your jobs to fail at 2AM on a Saturday, which is somewhat inconvenient.

Nodes are organized into partitions - for example, we have cpu_short, cpu_medium, cpu_long, and also fn_short, fn_medium, and fn_long. General-purpose CPU nodes (40 cores, ~380Gb memory) are in the cpu_* partitions, and fat nodes (64 cores, 1.5Tb or so of memory) are in the fn_* partitions. Partitions are a simple way to specify what kind of machine you want to run a job on. Here's some quick tips and commands to get started.

Some salient commands/things to know:

- Resource limits: ~360 CPUs, 1.6Tb of memory on the CPU partitions. After this, your jobs are held in queue
- As a rough rule, if your jobs use more than 9-10Gb of RAM per core/thread, they belong on the fat partition, not the cpu partition
- Show all my jobs
 - `squeue -me`
- List all the nodes/partitions

- o sinfo
- Submit a job (see sbatch documentation for explanations/other options)
 - o `sbatch -t dd-hh:mm:ss -e error_file -o log_file -p partition [other options]`
`jobscript.sh`
- How much file storage is available?
 - o `/gpfs/data/petljaklab/`
 - `mmquota -j data_petljaklab --block-size auto gpfs`
 - o `/gpfs/home/[my kerberos ID]`
 - `mmquota -u USERNAME --block-size auto gpfs:home`
- Other things to add as I remember/encounter them

Good Stewardship of BigPurple

The HPC team are not omniscient, omnipotent beings. We should try, wherever possible, to make sure we don't accidentally do stupid things to the cluster that make their lives difficult. This subsection details some things that we can/should do.

Checking /tmp/

Many programs will write to /tmp/ on the cluster and slowly fill that space

1. Wherever possible, specify the /tmp/ space for your programs explicitly, and ensure you delete the files you create after your script runs
2. I recommend having a cron job monitor and notify you. See the cron section for details on automating.
 - a. Handy script, copy it for yourself and use it to tell you if you're using >50% of /tmp and /tmp is >90% full:
`/gpfs/data/petljaklab/lculibrk_prj/scripts/check_tmp.bash`
 - b. To run this on all nodes, use a for loop:
 - i.

```
for i in $(scontrol show node | grep
NodeName=cn | sed 's/NodeName=//g' | sed 's/
.*//g'); do ssh $i /path/to/script.bash
```
 - c. Set up a cron job, e.g mine:
 - i.

```
0 0 * * * for i in $(scontrol show node | grep
NodeName=cn | sed 's/NodeName=//g' | sed 's/
.*//g'); do ssh $i
/gpfs/data/petljaklab/lculibrk_prj/scripts/chec
k_tmp.bash; done
```

Don't run things on the login nodes

When you can, don't run things on login nodes. HPC would rather you start up an interactive session, e.g. with `srun` ([see here](#)) to work interactively. That being said, it's a bit clunky compared to just running things, so while HPC might not like it, if you limit yourself to lighter, quick tasks on a login node for prototyping and the like, they will probably not come after you.

But wherever possible if something takes more than a few minutes it's probably better to either get an interactive session or to submit it as a job.

Peltjaklab partition

We (the Petljak lab) have a private node on BigPurple called "petljaklab", the node is cn-0057. It has 128 threads (64 physical cores) and 512Gb of RAM. This node is ideally used for running pipelines, but if there is free capacity you can submit jobs to it. Try to limit the jobs to utilizing ~4Gb of RAM per thread if they are not fast jobs, so we can effectively use the available compute.

In sbatch, you can specify partitions using "-p". Simply set the partition to petljaklab, or even a combination, such as "petljaklab,cpu_short", and the scheduler will do the rest.

Cron

<https://crontab.guru/>

<https://en.wikipedia.org/wiki/Cron>

Cron is an automator for running programs on a linux system. You can specify a job to run at specified times, automatically. It's useful for running tasks that need to be done at regular intervals, such as emailing people who are ignoring you, every hour, on the hour, forever, until they finally decide to reply to you.

Navigating vi

To set up a cron job on BigPurple, simply run "crontab -e". Be warned, because it will send you into a "vi" text editor, which basically nobody can figure out how to use for the first time. Luckily for you, I still have basically no comprehension of how to correctly use vim, so I can provide the basic usage instructions for people who also have no comprehension of vim.

To make changes to the file, you need to type "i" (this puts you into "insert" mode). Now you can text edit as you might expect. To leave insert mode, hit the escape key. To exit the program, exit insert mode with escape as previously mentioned, and type ":wq". Simple!

Also, enjoy this short compilation of memes about Vim being notoriously difficult to exit for first-time users.

https://www.reddit.com/r/vim/comments/m3zg46/a_memeworthy_compilation_of_images_from_a_group/

Adding cron jobs

I highly recommend adding a mailto line at the top of your crontab. This ensures that you get an email with the stderr of your job when it completes. For example my first line is:

```
"MAILTO = luka.culibrk@nyulangone.org"
```

Actual jobs are specified with the cron syntax. Firstly, you provide five values relating to the frequency of the job. Here's an example job that simply runs "ls ~/ " every day at 0:00:

```
0 0 * * * ls ~/
```

The meaning of the zeroes and asterisks can be found in the linked documentation at the start of the Cron section. Basically they are "hour minute day month day_of_week" - an asterisk denotes every time, so the above crontab line runs the command at 00:00 every day of every month.

Note that BigPurple has three login nodes; bigpurple-ln1, bigpurple-ln2, and bigpurple-ln3. Your cron jobs are specific to the login node you created them on, so if you set up cron on ln2, they will only be set up on ln2, not ln1 or ln3, so ensure you ssh back to ln2 when you need to modify them

OnDemand

Website: [ondemand](https://ondemand.nyu.edu/)

This is where you can launch interactive sessions. The most useful might be Rstudio.

At the top of the screen, click on "My Interactive Sessions". There will be a selection of apps on the left. Rstudio (New) is probably the most appropriate. You can safely use default settings on this page in many cases. However, in peak (most) hours there might be a wait to get a CPU job running on the cluster.

To mitigate wait times you can select less-frequented queues for your sessions.

Begin by running "sinfo". This will show you the status of all the partitions (queues):

```
(base) [culib101@bigpurple-ln3 variant_filtering_exploration]$ sinfo
PARTITION AVAIL TIMELIMIT NODES STATE NODELIST
prod*      up infinite 0 n/a
dev        up infinite 0 n/a
cpu_dev    up 4:00:00 2 mix cn-[0004-0005]
cpu_dev    up 4:00:00 2 idle cn-[0002-0003]
cpu_short  up 12:00:00 29 mix cn-[0004-0007,0010,0012,0014,0016-0020,0023,0028,0031-0033,0035-0043,0045-0047]
cpu_short  up 12:00:00 14 alloc cn-[0008-0009,0011,0013,0015,0021,0024-0027,0029-0030,0034,0044]
cpu_medium up 5-00:00:00 29 mix cn-[0004-0007,0010,0012,0014,0016-0020,0023,0028,0031-0033,0035-0043,0045-0047]
cpu_medium up 5-00:00:00 14 alloc cn-[0008-0009,0011,0013,0015,0021,0024-0027,0029-0030,0034,0044]
cpu_long   up 28-00:00:0 6 mix cn-[0042-0043,0045-0047,0050]
cpu_long   up 28-00:00:0 1 alloc cn-0044
fn_short   up 12:00:00 4 mix fn-0001,fn2-[0001-0003]
fn_short   up 12:00:00 2 idle fn-[0002-0003]
fn_medium  up 5-00:00:00 4 mix fn-0001,fn2-[0001-0003]
fn_medium  up 5-00:00:00 2 idle fn-[0002-0003]
fn_long    up 28-00:00:0 4 mix fn-0004,fn2-[0002-0004]
fn_long    up 28-00:00:0 2 idle fn-[0002-0003]
gpu4_dev   up 4:00:00 2 idle gn-[0002-0003]
gpu4_short up 12:00:00 9 mix gn-[0006,0008-0009,0015-0020]
gpu4_short up 12:00:00 3 alloc gn-[0007,0010-0011]
gpu4_medium up 3-00:00:00 9 mix gn-[0006,0008-0009,0015-0020]
gpu4_medium up 3-00:00:00 3 alloc gn-[0007,0010-0011]
gpu4_long  up 28-00:00:0 2 mix gn-[0020,0023]
gpu4_long  up 28-00:00:0 1 alloc gn-0024
gpu8_short up 12:00:00 1 down* gpu-0002
gpu8_short up 12:00:00 2 mix gpu-[0001,0003]
gpu8_short up 12:00:00 2 alloc gpu-[0004-0005]
gpu8_medium up 3-00:00:00 1 down* gpu-0002
gpu8_medium up 3-00:00:00 2 mix gpu-[0001,0003]
gpu8_medium up 3-00:00:00 2 alloc gpu-[0004-0005]
gpu8_long  up 28-00:00:0 1 mix gpu-0007
gpu8_long  up 28-00:00:0 3 alloc gpu-[0004-0006]
a100_dev   up 4:00:00 5 mix a100-[4002-4006]
a100_short up 3-00:00:00 1 down* a100-4019
a100_short up 3-00:00:00 23 mix a100-[4004-4011,4014-4018,4021-4022,4024,4027-4033]
a100_short up 3-00:00:00 4 idle a100-[4020,4023,4025-4026]
a100_long  up 28-00:00:0 15 mix a100-[4030-4034,4036-4039,4041-4045,4047]
a100_long  up 28-00:00:0 1 alloc a100-4035
a100_long  up 28-00:00:0 2 idle a100-[4040,4046]
data_mover up 3-00:00:00 2 mix dmn-[0002-0003]
data_mover up 3-00:00:00 2 idle dmn-[0001,0004]
```

Notice that while cpu queues are all mix/alloc (mostly used), the fn_ partitions have some idle nodes. I.e. 1/3 of the fn_short queue is completely idle. This is wasteful, so we should help to ensure that UltraViolet is as congested as possible by using the fn_ nodes too.

Under your session options, you can select “Advanced” for the instance type. Select your desired partition, e.g. “fn_short” and put sensible requests for resources. For the fat nodes, allegedly you need to request at least 8Gb of memory per core. If you ask for an ineligible amount of resources then it will either go to the cpu partitions or the job will be stuck in queue forever. If it's taking suspiciously long then you'll want to investigate e.g using squeue and sacct to know why.

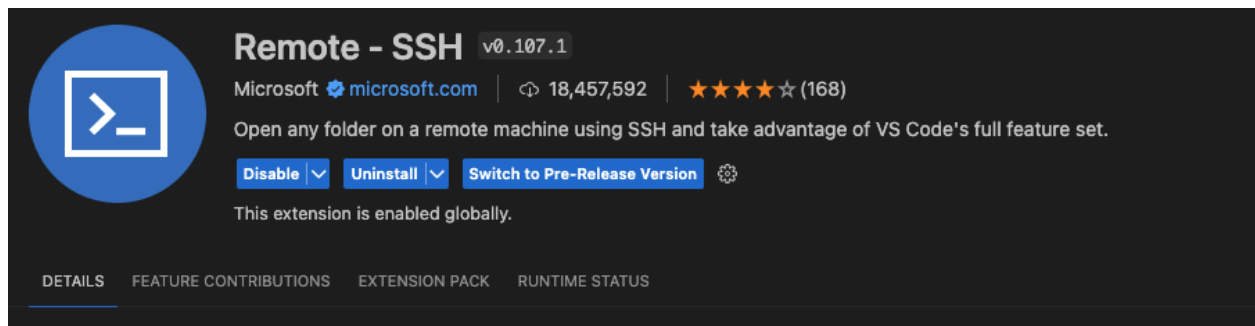
VSCode and VSCode Remote

[Visual Studio Code](#) is an incredibly powerful and user-friendly application for writing code. It's developed by Microsoft and ubiquitous/good enough that even Google software engineers couldn't be made to swap from VSCode to Google's in-house copycat. You don't have to use it if you have your own solutions that work for you, but if you don't know what you're doing then VSCode is a good place to start. This section will walk you through setting up VSCode remote so you can work with files on BP/UltraViolet.

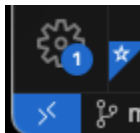
When you open VSCode there should be a menu on the left with an icon like this:



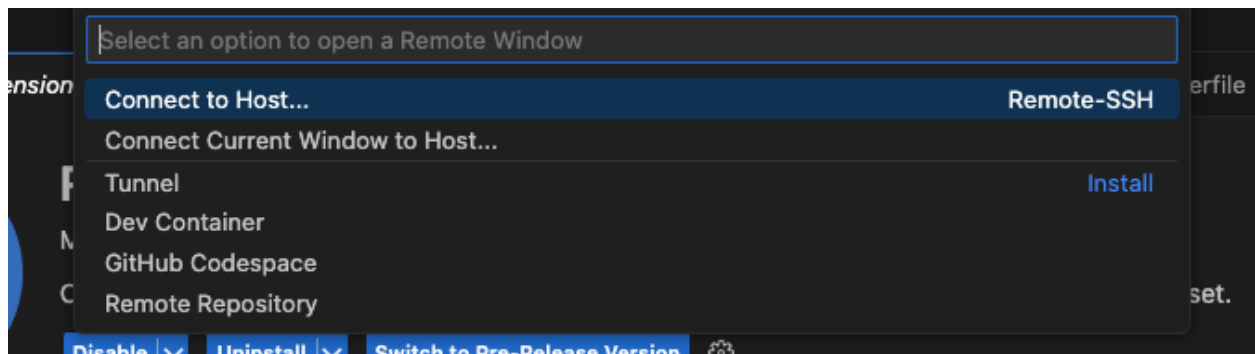
This is the extensions menu. Click on it and search for “remote”. You’re looking for something like this:



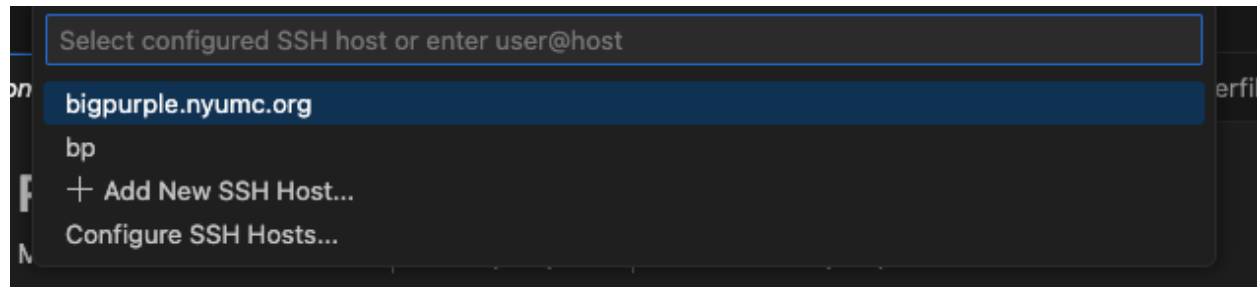
Install it, then click the little blue icon on the extreme bottom left of the VSCode window:



This will activate a search/text box on the top middle:



Click the Connect to Host button. You'll see some new options, and will not be identical to mine below. But similar.



Click on Configure SSH Hosts and select the first offered file (might be /Users/blah/blah/.ssh/config or something similar).

Now you'll have a text file open to modify. Add a few lines, like shown below:

```
.ssh > ≡ config
1  Host bigpurple.nyumc.org
2    HostName bigpurple.nyumc.org
3    User culibl01
4
```

Clearly you should replace my username (culibl01) with your own kerberos ID. It is case sensitive. Save the file and close it. Now follow the previous instructions beginning from the small blue icon, but now you should have bigpurple.nyumc.org as an option once you click on connect to host. Click on it - it should open a new window. Now you'll need to enter your password. Congratulations, now you can edit/work with text file, view images, view PDFs (with an extension), and more on the cluster!

R Tips/Tricks

ggplot and tidyr/dplyr cheat sheets:

<https://www.rstudio.com/wp-content/uploads/2015/02/data-wrangling-cheatsheet.pdf>

<https://www.maths.usyd.edu.au/u/UG/SM/STAT3022/r/current/Misc/data-visualization-2.1.pdf>

My R is slow!

- R being slow is somewhat of a misconception. There's a lot of overlap between self-taught/new programmers and R users, which leads to a lot of cases of inefficient code. R is actually ludicrously fast in a lot of use cases.
- For loops:
 - Use the [apply](#) functions instead if possible
 - If you absolutely need to use a loop, keep some concepts in mind:
 - If you need to append to an object (vector or list), make an empty object of the size you're making first. If you're doing 100 iterations of something in a for loop and saving each iteration to a list, make an empty list like `resultlist = rep(list(), times = 100)`. Appending is a costly process
- The above is a common example. See the [R inferno](#) for more examples. It's also a funny document. Excerpt:

Once we had crossed the Acheron, we arrived in the first Circle, home of the virtuous pagans. These are people who live in ignorance of the Floating Point Gods. These pagans expect

`.1 == .3 / 3`

to be true.

If you care about performance you probably don't want to be using tidyverse. If you're doing table operations (joins, etc) on gigantic dataframes then you probably want to use the `data.table` package. In many circumstances it's even faster than domain-specific database frameworks like Spark:

- <https://h2oai.github.io/db-benchmark/>
- `Data.table` has some of its own idiosyncracies much like tidyverse so feel free to ask your resident `data.table` stan for help (Luka)

Example on spreading a ~3 sample concatenated VAF table long-to-wide (450k to 150k rows):

```
> t1 = Sys.time()
> spread_tbl = dcast(parents_vars, formula = CHROM + POS ~ sample, value.var = "vaf")
> t2 = Sys.time()
> print(t2-t1)
Time difference of 0.104784 secs
> library(tidyverse)
> t1 = Sys.time()
> spread_tbl = parents_vars %>% select(CHROM, POS, sample, vaf) %>% spread(sample, vaf)
> t2 = Sys.time()
> print(t2-t1)
Time difference of 0.2065587 secs
```

The time can add up if you're doing many many joining operations, or if you're using gigantic tables. ~2x time differences can be substantial sometimes, so keep this in mind.

Petljakdb

We have a mySQL server running on BigPurple for storing/querying metadata. While more rigorous documentation is coming, here is a quick primer on how to access and use it.

Host: "db"

Port: 33100

First, you will need a database account (ask Luka)

mySQL stores data in tables that can be linked to one another. Our data are stored as follows:

1. There is a "studies" table for each project/study. IDs on the filesystem are prefixed MPP (**M**ia **P**etljak **P**roject).
2. Each sample is stored in the "samples" table. Prefix MPS
3. Each run (ie. sequencing run) is stored in the "runs" table. Prefix MPR
4. Each analysis is stored in the "analyses" table. Prefix MPA

We also have a cells table that stores each cell line we've sequenced to allow for queries across studies for specific lines.

How to access:

1. Load the mariadb module with "module load mariadb"
2. `mysql -u "username" -p "password" -h host`
3. You are now in a mySQL command line interface

mySQL documentation abounds, and chatGPT can be a good help for helping to write queries. Some examples (trailing semicolon is important):

Show all the projects/samples/runs (sub in {tablename}):

```
SELECT * FROM petljakdb.{tablename};
```

Describe a table:

```
DESCRIBE petljakdb.{tablename};
```

Show all samples from study MPP000003:

```
SELECT * FROM petljakdb.samples WHERE study_id=3;
```

Show all the BC-1 cells, regardless of study:

```
SELECT * FROM samples where cell_id = (select id from cells where rname = "BC-1");
```

Show the WGS bam/cram paths for a study:

```
SELECT id,analysis_dir FROM petljakdb.analyses WHERE analysis_type="WGS_MERGE_BAM" AND analysis_complete='True' AND studies_id=1;
```

If you want to dump a query result to a text file, you can run this from bash:

```
mysql -u "username" -p "password" -h host -B --database=petljakdb --execute 'YOUR QUERY HERE' > file.txt
```

Finally, you can query the database from Rstudio running on BigPurple:

```
library(RMySQL)
con = RMySQL::dbConnect(MySQL(), user = "username", password = "password", host =
"host", dbname = "petljakdb")
rs = dbSendQuery(con, "SELECT rname FROM samples WHERE study_id=3;")
dbFetch(rs)
```